0.1 MixStyle

0.1.1 インポート

```python
import torch
from torch import nn
from torch.distributions.beta import Beta
```

0.1.2 主な関数

```python
def calc_weighted_stats(x, attn_map):
    """Attentionマップを重みとして平均と標準偏差を計算.

    x: [Batch, Channel, Time, Freq]
    attn_map: [Batch, Channel, 1, Freq]
    """
    EPSILON = 1e-6
    attn_sum = attn_map.sum(dim=(2), keepdim=True) + EPSILON

    # 1. 重み付き平均
    mean = (x * attn_map).sum(dim=(2), keepdim=True) / attn_sum

    # 2. 重み付き分散
    var = ((x - mean) ** 2 * attn_map).sum(dim=(2), keepdim=True) / attn_sum
    std = torch.sqrt(var + EPSILON)

    return mean, std


def mix_style(x, attn_map, use_attn):
    """バッチ間で統計量を混合し，データ拡張を実施.

    Args:
        x: Input features (batch_size, n_channels, n_frames, n_freq)

    Returns:
        Augmented features with mixed statistics
```

```
    """
    AUGMENTATION_PROB = 0.5
    if torch.rand(1).item() > AUGMENTATION_PROB:
        return x
    EPSILON = 1e-6

    if use_attn == "able":
        x_mean, x_std = calc_weighted_stats(x, attn_map)

    x_mean = x.mean(dim=(2), keepdim=True)
    x_var = x.var(dim=(2), keepdim=True)
    x_std = (x_var + EPSILON).sqrt()

    x_normed = (x - x_mean) / x_std

    # バッチ内で異なる統計量を得る
    B = x.size(0)
    perm = torch.randperm(B, device=x.device)
    mu2, sig2 = x_mean[perm], x_std[perm]

    # Beta 分布に基づき混合
    BETA_ALPHA = 0.1
    lam = Beta(BETA_ALPHA, BETA_ALPHA).sample((B, 1, 1, 1))
    lam = lam.to(device=x.device)

    mixed_mean = lam * x_mean + (1 - lam) * mu2
    mixed_std = lam * x_std + (1 - lam) * sig2

    return x_normed * mixed_std + mixed_mean
```

0.1.3 クラス

```
# Main MixStyle Classes
class BasicMixStyleWrapper(nn.Module):
    """Wrapper for pure MixStyle without attention mechanism.

    This is used for baseline experiments that only apply
    basic MixStyle augmentation without frequency-wise attention.
```

```
    """

    def __init__(self, channels=None, **kwargs):
        """Initialize BasicMixStyleWrapper.

        Args:
            channels: Not used, but kept for API compatibility
            **kwargs: Not used, but kept for API compatibility

        """
        super().__init__()
        # No parameters needed - just wraps mix_style function

    def forward(self, x):
        """Forward pass applying pure MixStyle.

        Args:
            x: Input features (Batch, Channels, Frame, Frequency)

        Returns:
            Output features with pure MixStyle applied

        """
        return mix_style(x, None, None)


class FrequencyAttentionMixStyle(nn.Module):
    """CNN-based frequency attention MixStyle.

    Args:
        channels: Number of input channels
        **kwargs: Configuration options
            - attn_type: Attention network architecture
                * "default": Shallow 2-layer CNN

    """

    def __init__(self, channels, **kwargs):
        super().__init__()
        self.channels = channels

```

```python
        # Configuration options
        self.attention_type = kwargs.get("attn_type", "default")

        # Calculate intermediate channel dimensions
        specific_channels = channels if channels == 1 else channels // 2

        # Build attention network
        self.attention_network = self._build_attention_network(
            channels,
            specific_channels,
            self.attention_type,
        )

    def _build_attention_network(self, in_channels, mid_channels, attn_type):
        """Build attention network based on specified architecture.

        Args:
            in_channels: Input channel dimension
            mid_channels: Intermediate channel dimension

        Returns:
            Constructed attention network module

        """
        if attn_type == "default":
            network = nn.Sequential(
                nn.Conv1d(in_channels, mid_channels, kernel_size=3, padding=1),
                nn.ReLU(),
                nn.Conv1d(mid_channels, in_channels, kernel_size=1),
            )
            # Initialize final layer bias to 1.0 to avoid sigmoid saturation at 0.5
            # sigmoid(1.0) = 0.73, providing bias towards mixed features
            nn.init.constant_(network[-1].bias, 1.0)
            return network

    def forward(self, x):
        """Forward pass.

        Args:
            x: Input features (Batch, Channels, Frame, Frequency)

```

```
        Returns:
            Output features with frequency-wise MixStyle applied

        """
        x_avg = x.mean(dim=2)  # (B, C, F)
        attn_logits = self.attention_network(x_avg)

        attn_weights = torch.sigmoid(attn_logits).unsqueeze(-2)  # (B, C, 1, F)
        x_mixed = mix_style(x, attn_weights, "able")
        return x + x_mixed
```